



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Django Templates – Theory and Practical Notes

1. Introduction to Django Templates

Django is a Python web framework used to build dynamic web applications. When we create a website using Django, we usually need to display web pages to the user. These web pages are written using HTML.

A Django Template is an HTML file that Django uses to display content in the browser. Unlike a normal static HTML file, a Django template can also receive and display dynamic data from Python.

For example, suppose we want to display the name of a student on a webpage. Instead of writing the student's name directly inside HTML, we can send the name from a Django view to the template. The template can then display that name dynamically.

Django mainly follows the MVT architecture, which stands for:

- Model – Manages data and database operations.
- View – Handles application logic and requests.
- Template – Handles what the user sees in the browser.

In simple words, the View prepares the data, and the Template displays that data.

2. Creating a Django Project

Before working with Django Templates, we first need to create a Django project.

Make sure Python is installed on your computer. We also need to install Django.



Open the terminal or command prompt and run:

```
pip install django
```

To check whether Django is installed correctly, run:

```
django-admin --version
```

If Django is installed successfully, the terminal will display the installed Django version.

Now create a new Django project. For example, we will create a project named myproject.

```
django-admin startproject myproject
```

Move inside the project folder:

```
cd myproject
```

The project will contain files such as:

```
myproject/
├── manage.py
├── myproject/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
```

The manage.py file is used to perform different Django commands, such as running the development server and creating applications.

The settings.py file contains the configuration of the Django project.

The urls.py file is used to manage the URLs or routes of our website.

3. Running the Django Project

After creating the project, we can start Django's development server.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Run:

```
python manage.py runserver
```

On some systems, you may need to use:

```
python3 manage.py runserver
```

After running this command, Django will start a local development server. Normally, the website can be opened at:

```
http://127.0.0.1:8000/
```

Open this address in your web browser.

If Django is working correctly, you will see the default Django welcome page.

The development server is useful while building and testing a Django application. Whenever we make changes to our code, Django usually reloads the server automatically.

To stop the server, press:

```
Ctrl + C
```

in the terminal.

4. Creating a Django Application

A Django project and a Django application are different concepts.

The project represents the complete website or web system. An application represents a specific part or feature of that project.

For example, an e-commerce project may contain separate applications for products, customers, orders, and payments.

For our example, we will create an application called myapp.

Run:

```
python manage.py startapp myapp
```



Django will create a new folder called myapp.

The structure will look similar to this:

```
myproject/
├── manage.py
├── myproject/
│   ├── settings.py
│   └── urls.py
├── myapp/
├── migrations/
├── admin.py
├── apps.py
├── models.py
├── tests.py
└── views.py
```

The views.py file is important for our templates because we will create Python functions inside this file that render our HTML templates.

5. Registering the Django Application

After creating an application, it is good practice to register it in the Django project.

Open:

```
myproject/settings.py
```

Find the `INSTALLED_APPS` section and add myapp:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'myapp',
]
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



]

Now Django knows that myapp is part of our project.

6. Creating the Templates Folder

Django templates are normally stored inside a folder named templates.

In this example, we will create a project-level templates folder.

Create a folder called templates in the same directory where manage.py is located.

The project structure will now look like this:

```
myproject/
├── manage.py
├── templates/
├── myapp/
└── myproject/
```

Inside the templates folder, create an HTML file called:

home.html

The structure becomes:

```
myproject/
├── manage.py
├── templates/
│   └── home.html
├── myapp/
│   └── views.py
├── myproject/
├── settings.py
└── urls.py
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



The home.html file will be our first Django template.

7. Configuring the Templates Folder

Django needs to know where our templates are stored.

Open:

```
myproject/settings.py
```

Find the TEMPLATES configuration.

By default, it looks similar to this:

```
TEMPLATES = [
{
    'BACKEND': 'django.template.backends.django.DjangoTemplates',
    'DIRS': [],
    'APP_DIRS': True,
    'OPTIONS': {
        'context_processors': [
            ...
        ],
    },
},
]
```

We need to modify the DIRS option:

```
'DIRS': [BASE_DIR / 'templates'],
```

The configuration will look similar to:

```
TEMPLATES = [
{
    'BACKEND': 'django.template.backends.django.DjangoTemplates',
    'DIRS': [BASE_DIR / 'templates'],
    'APP_DIRS': True,
    'OPTIONS': {
        'context_processors': [
            ...
        ],
    },
},
]
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
},  
},  
]
```

BASE_DIR represents the main directory of our Django project.

By adding:

```
BASE_DIR / 'templates'
```

we are telling Django to search for HTML template files inside the templates folder.

8. Creating Our First Django Template

Now open:

```
templates/home.html
```

Add basic HTML code:

```
<!DOCTYPE html>  
<html>  
<head>  
<title>Django Templates</title>  
</head>  
<body>  
<h1>Welcome to Django</h1>  
<p>This page is displayed using a Django Template.</p>  
</body>  
</html>
```

At this stage, this is a simple HTML page. However, Django will be responsible for loading and displaying this page.

To display the template, we need to create a View.

9. Creating a Django View



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



A Django View receives a request from the browser and returns a response.

Open:

```
myapp/views.py
```

Add the following code:

```
from django.shortcuts import render
def home(request):
    return render(request, 'home.html')
```

Here, home is a view function.

The request parameter contains information about the request received from the browser.

The render() function tells Django to load the home.html template and return it to the browser.

In simple words, when the home view runs, Django finds home.html, processes it, and displays the final HTML page.

10. Connecting the View with a URL

Creating a view is not enough. We also need to create a URL so that the browser can access that view.

Open:

```
myproject/urls.py
```

By default, it may contain:

```
from django.contrib import admin
from django.urls import path
urlpatterns = [
    path('admin/', admin.site.urls),
]
```

Import the home view:

```
from myapp.views import home
```

Then add a new URL path:



```
from django.contrib import admin
from django.urls import path
from myapp.views import home
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', home, name='home'),
]
```

The empty string:

```
''
```

represents the homepage of the website.

Now run the server:

```
python manage.py runserver
```

Open:

```
http://127.0.0.1:8000/
```

Django will follow this flow:

Browser Request

↓

URL

↓

View

↓

Template

↓

HTML Response

↓

Browser

The browser requests the homepage. Django checks `urls.py` and finds the home view. The home view then renders `home.html`, and the final webpage is returned to the browser.

This is the basic working process of Django Templates.

11. Passing Data from a View to a Template



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



One of the main advantages of Django Templates is the ability to display dynamic data.

Suppose we want to send a student's name from Python to HTML.

Modify the home view:

```
def home(request):  
    context = {  
        'name': 'Ali'  
    }  
    return render(request, 'home.html', context)
```

Here, context is a Python dictionary.

The key:

```
'name'
```

becomes a variable that we can use inside the template.

Now open home.html:

```
<h1>Welcome {{ name }}</h1>
```

When Django processes this template, it replaces:

```
{{ name }}
```

with:

```
Ali
```

The browser will display:

Output:

```
Welcome Ali
```

The double curly brackets `{{ }}` are used to display variables in Django Templates.

12. Using Multiple Template Variables

We can send multiple values from the view.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



For example:

```
def home(request):  
    context = {  
        'name': 'Ali',  
        'course': 'Django',  
        'semester': 5  
    }  
    return render(request, 'home.html', context)
```

Inside the template:

```
<h1>Student Information</h1>  
<p>Name: {{ name }}</p>  
<p>Course: {{ course }}</p>  
<p>Semester: {{ semester }}</p>
```

Django will automatically replace the template variables with the values received from the view.

This allows us to create dynamic webpages where the displayed information can change depending on the data provided by the application.

13. Django Template Tags

Django Templates also provide Template Tags.

Template tags allow us to add simple programming logic inside HTML templates.

Template tags use:

```
{% %}
```

If Condition

For example, we can use an if condition:

```
{% if name %}  
<h1>Welcome {{ name }}</h1>  
{% endif %}
```

For Loop



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



We can also use loops.

Suppose the view sends a list:

```
def home(request):  
    context = {  
        'students': ['Ali', 'Ahmed', 'Sara']  
    }  
    return render(request, 'home.html', context)
```

Inside the template:

```
<h2>Students</h2>  
{% for student in students %}  
<p>{{ student }}</p>  
{% endfor %}
```

Django will repeat the paragraph for every student in the list.

The output will be:

Output:

Ali
Ahmed
Sara

Template tags are useful because they allow us to control how data is displayed without writing Python code directly inside HTML.

14. Template Inheritance

When creating a website, many pages have the same structure.

For example, every page may contain the same navigation bar, header, and footer. Instead of writing the same HTML code again and again, Django provides Template Inheritance.

We can create a main template called:

`base.html`

For example:



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
<!DOCTYPE html>
<html>
<head>
<title>My Django Website</title>
</head>
<body>
<h1>My Website</h1>
{% block content %}
{% endblock %}
<footer>
<p>Copyright 2026</p>
</footer>
</body>
</html>
```

The following code creates a content area:

```
{% block content %}
{% endblock %}
```

Other templates can use this main template.

For example, home.html:

```
{% extends 'base.html' %}
{% block content %}
<h2>Welcome to the Home Page</h2>
<p>This content comes from home.html.</p>
{% endblock %}
```

The line:

```
{% extends 'base.html' %}
```

tells Django that home.html should use the structure of base.html.

Only the content inside:

```
{% block content %}
```

will be different for each page.

Template inheritance helps us avoid repeating the same HTML code and makes the project easier to maintain.



15. Complete Django Template Workflow

The complete process can be understood as:

- User opens a URL
- Django checks urls.py
- URL calls a View
- View prepares data
- View sends data to a Template
- Template processes HTML and dynamic data
- Final HTML is returned to the browser

For example:

```
# views.py
def home(request):
    context = {
        'name': 'Ali'
    }
    return render(request, 'home.html', context)
```

The URL connects to the view:

```
# urls.py
path('', home, name='home')
```

The template displays the data:

```
<!-- home.html -->
<h1>Welcome {{ name }}</h1>
```

The final result displayed in the browser is:

Output:

Welcome Ali



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



This is the basic concept behind Django Templates. Templates allow Django applications to combine Python application logic with HTML presentation while keeping both parts separate. This separation makes Django applications easier to organize, develop, and maintain.